RythuSaathi

Your Intelligent Farming Companion.

Architecture & Implementation Plan

A practical, evidence-driven design for a multi-agent agricultural advisory system, scoped for a solo developer and a ten-day delivery window.

Planning baseline
All-India ambition, extensible crop registry, web + CLI, Microsoft Agent Framework, shared advisory blackboard, official-source RAG.

Prepared 02 November 2026

Planning only | No application files or dependencies were created or changed.

1. Executive recommendation

Build a single-process Python application with six separately identifiable agents operating on one per-case **AdvisoryBoard**. Keep the workflow fixed and sequential for the first release. Each agent calls real Python tools for validation, retrieval, parsing, or calculation; the coordinator checks evidence and safety before rendering the farmer-facing result.

The system should be evidence-first: every claim points to a source or is marked unavailable/uncertain. External data access is attempted only through documented provider adapters. Demonstration fixtures are explicitly labeled as fixtures, never as live observations.

Microsoft Agent Framework is the required orchestration layer. Its graph workflow model supports explicit steps and execution events; Python functional workflows are documented as experimental, so the graph API is the safer fit for this assignment. Workflow documentation.

Ten-day MVP target

One complete case runs Intake -> Weather -> Crop -> Market -> Scheme -> Coordinator; exposes the run through CLI and a small web form; uses real sources where access is verified and clearly labeled fixtures otherwise; and returns field action, market timing, relevant scheme guidance, uncertainty, safety notices, citations, and the qualified-professional disclaimer.

Workspace inspection

The inspected workspace is empty: no source files, README, Python modules, or AGENTS.md were present. Git was not initialized. No project files were changed during planning.

2. Architecture overview

The board is the shared case record and source of truth. Agents append findings through a board API that enforces agent write scope. The fixed workflow runs agents sequentially initially; concurrency can be considered later after the board mutation and trace behavior are proven.

```
Web form / CLI -> Intake -> Weather -> Crop -> Market -> Scheme -> Coordinator
Each agent reads and appends to the same per-case AdvisoryBoard.
```

Separation of responsibilities: framework orchestration schedules agents; Python tools perform deterministic operations; the board stores case evidence and trace; the LLM interprets retrieved context and words a bounded advisory; the coordinator applies completeness and safety gates.

Design constraints

- No optional Risk Alert Agent. Mandatory threat detection and safety checks live in deterministic tools and coordinator gates.
- No voice input/output or multilingual generation during the initial ten days. Keep a language preference field only if useful for future compatibility.
- No unsupported guarantees or exact action claims without evidence. Do not infer missing farmer facts.
- One process and one board per case keep the system understandable and easy to demonstrate.

3. Recommended repository structure

Keep the implementation compact; the tree below is a target organization rather than a requirement to create empty folders.

```
rythusaathi/
|-- README.md (maximum two pages)
|-- pyproject.toml | .env.example | .gitignore
|-- src/rythusaathi/
| |-- config.py | board.py | workflow.py
| |-- schemas/{case,evidence,board,advisory}.py
| |-- agents/{intake,weather,crop,market,scheme,coordinator}.py
| |-- tools/{locations,weather,crops,market,schemes}.py
| |-- data/{crop_registry.json,locations.json,knowledge/}
| |-- rag/{ingest,search}.py
| |-- interfaces/{cli,web}.py
| `-- logging/execution_log.py
|-- notebooks/demo.ipynb
|-- docs/architecture.mmd
|-- logs/ (append-only run records)
`-- tests/{test_intake,test_board,test_workflow_scenarios}.py
```

4. Blackboard and data contracts

Create one Pydantic-validated **AdvisoryBoard** per case. Its top-level sections are: case, weather, crop_guidance, market, schemes, risks, advisory, and trace. Sections hold typed records; agent results are appended, not replaced.

Model / section	Required contents
Case	case_id; crop; state; district; crop_stage; farmer observations; optional language preference field
EvidenceRecord	evidence_id; agent_id; source name/reference; retrieved_at; observed_at when applicable; units; quality_status; confidence when meaningful; typed payload
Weather	forecast evidence; rainfall; warning/alert evidence; provider status
Crop guidance	stage guidance; supporting evidence IDs; weather-stage assessment; limitations
Market	mandi and commodity; minimum/maximum/modal prices; units; observation dates; calculated trend
Schemes	scheme identity; passages with page references; criterion-level eligibility evidence
Risks / advisory	risk flags; missing or stale evidence; field action; market timing; scheme guidance; uncertainty; sources; disclaimer
Trace	agent/tool identity; start/end timestamp; status; error category; duration; case ID

Use explicit status values: available, unavailable, stale, failed, unsupported. Keep statuses separate from quality/confidence. Each write should go through append(agent_id, section, item), validate the agent’s allowed section, then add a trace event. Do not expose raw mutable lists to agents.

5. Agent contracts

Agent	Reads	Writes
Intake	Raw request + registries	Validated Case; location resolution evidence; validation errors / missing-input flags
Weather	Validated case	Forecast and warning records; stale/unavailable/failed flags
Crop	Case + weather + crop corpus	Stage guidance; weather-stage risk interpretation; source links and limitations
Market	Case + commodity mapping	Dated mandi observations; price trend; quality and freshness flags
Scheme	Case + scheme corpus	Retrieved passages; criterion-level met / unmet / unknown evidence
Coordinator	Whole board	Evidence completeness result; safety checks; final advisory; uncertainty and sources

6. Tool contracts and implementation responsibilities

Agent	Meaningful Python tools	What each tool actually does
Intake	validate_crop(crop_name, registry) resolve_location(state, district, index)	Normalize and validate against controlled registries; surface ambiguity instead of guessing.
Weather	fetch_forecast(location, provider) fetch_weather_alerts(location, provider)	Call the configured adapter; parse and timestamp results; preserve units/provenance; return explicit failure states.
Crop	retrieve_crop_guidance(crop, stage, location, corpus) evaluate_weather_stage_risk(guidance, weather)	Retrieve cited local guidance; compare only supported weather factors and return limitations.
Market	fetch_mandi_prices(commodity, district, range, provider) calculate_price_trend(observations)	Retrieve dated observations; compute a stated, reproducible trend; do not guarantee a best selling date.
Scheme	search_scheme_documents(query, index) check_scheme_eligibility(criteria, farmer_evidence)	Search page-aware passages; compare available facts; label each criterion met, unmet, or unknown.
Coordinator	validate_evidence_completeness(board) render_farmer_advisory(board)	Check required evidence and safety blockers; produce only claims tied to the board and cite evidence IDs.

Every working agent invokes at least two such functions in its ordinary path. Each tool has a distinct operational purpose; none exists just to satisfy a tool-count requirement.

7. Microsoft Agent Framework strategy

- Create six distinct framework agents and a graph workflow with explicit sequential steps.
- Bind each agent tools to that case one AdvisoryBoard instance. Keep business logic in ordinary Python modules for independent tests.
- Consume framework workflow/executor events and write agent/tool start, finish, and failure entries to the append-only trace.
- Keep provider setup behind configuration. Do not require credentials for deterministic test/demo runs.
- Pin the framework only after a Python 3.11 integration spike confirms import, tool registration, graph execution, and event streaming. Do not switch to Semantic Kernel without an evidenced blocker and user approval.

The official docs describe agent construction, function tools, and graph workflows. They also caution that shared application state needs synchronization before concurrent execution. Sequential execution avoids that extra risk in the MVP. Function tools | Workflow execution.

8. Dependency recommendations

Package	Recommended line / role	Planning note
Python	3.11	Project requirement; supported by candidate packages below.
agent-framework	Microsoft package; pin after smoke test	Core orchestration. Exact release should be checked against Python 3.11 and current docs before locking.
pydantic	v2 (2.14.x current line)	Schemas and validation; PyPI declares Python 3.10+.
fastapi	Current compatible release	Thin web interface; current release supports Pydantic v2 and Python 3.10+.
httpx	0.28.x	HTTP client for source adapters; supports Python 3.8+.
pytest	9.x	Automated deterministic scenarios; supports Python 3.10+.
pypdf	Current compatible release	Extract official PDF content.
scikit-learn	Optional current compatible release	TF-IDF lexical search for small RAG corpus.
jupyter / ipykernel	Current compatible releases	Runnable demonstration notebook.

This is a recommendation, not a tested lockfile. No dependencies were installed during planning. Add only provider SDKs that the selected model actually needs. Use .env for secrets and commit .env.example only.

9. Data source feasibility

Source	What is verified / uncertain	Recommended implementation
OGD India / AGMARKNET	Official catalog describes daily wholesale minimum, maximum, and modal prices generated through AGMARKNET and exposes a Catalog API link. Query access, authentication, rate limits, and freshness still need an implementation spike.	Check resource API requirements; normalize commodity names; preserve market, date, units, and source status. Fixtures if access fails.
IMD	Official API reference lists city forecasts, district warnings, rainfall and other products. Official API portal is an access gateway with registration/login. Anonymous access and district mapping suitability are not assumed.	Verify authentication, response format, mappings, issue/validity times, and units. Begin with documented district warning and forecast services.
ICAR / State agricultural universities	Institutional crop publications are available, but no single open national crop-stage API was verified in this review. Documents vary by region and revision date.	Curate a narrow set of official, cited crop-stage references and record publisher, date, geography, page, and limitations.
Official scheme documents	Official portals and PDFs can support grounded retrieval; scheme rules and application status change over time. Retrieval cannot certify a farmer eligibility.	Use a small official document set, page-level citations, retrieval dates, criterion checks, and unknown where evidence is insufficient.

Source modes must be visible on every record: live, cached, historical, or demo_fixture. Never call sample records live. Source checks: OGD mandi dataset | IMD API reference | IMD data portal.

10. RAG architecture

Use a small, auditable retrieval pipeline: manually curate official PDFs; extract text while retaining title, publisher, URL, publication date, retrieval date, and page number; split into page-aware chunks; index with TF-IDF lexical search; return top passages and metadata to the Scheme Agent; render citations. Keep scheme criteria structured where the document supports it.

A vector database and embedding pipeline are unnecessary for the first ten days. Re-ingest manually and mark replaced documents as superseded. If evaluation shows lexical search misses relevant passages, reconsider embeddings as a later decision.

11. Testing strategy

Scenario	Expected result
Complete case	All six agents run; board has source evidence; final advisory includes field action, market timing, scheme, source references and disclaimer.
Unavailable / stale data	Provider failure and old observations remain explicit; risks are appended; dependent claims are withheld or qualified.
Severe weather warning	Safety notice is prominent; ordinary action is appropriately limited; disclaimer is included.
Invalid / ambiguous intake	Bad crop or uncertain district is surfaced; no false location-specific downstream guidance.
Blackboard append-only	Independent agent findings remain; unauthorized writes and replacements are rejected.

Use fixtures and no live network calls in automated tests. The first three scenarios satisfy the minimum of three while covering end-to-end, failure, and safety behavior.

12. Ten-day milestones and acceptance criteria

Day	Deliverable	Acceptance criteria
1	Contracts + scope	Board schemas, write ownership, status vocabulary, demo crop/location, source plan agreed.
2	Board + framework	Append-only board and trace work; six identities/workflow spike runs on Python 3.11.
3	Intake + Weather	Input validation works; documented IMD call attempted; fixture and failure paths represented.
4	Crop + Market	Cited crop retrieval; explicit weather assessment; dated price trend with units.
5	Scheme RAG	Official documents ingested with page metadata; retrieval and unknown eligibility states work.
6	Coordinator + safety	Completeness/safety gates; field, market, scheme, uncertainty, citations, disclaimer in output.
7	Web + CLI	Both interfaces invoke the same workflow; API/model errors yield understandable partial results.
8	Tests + failures	At least three deterministic scenarios cover complete, unavailable/stale, and safety cases.
9	Notebook + docs	Notebook shows execution trace; diagram renders; README is no more than two pages.
10	Submission rehearsal	Clean instructions work; offline fixture demo is labeled; live source status is accurate; no unsupported claims.

13. Minimum viable demonstration

One CLI or web request for one curated crop, district, and stage traverses every agent on the same board. Weather and mandi use verified live access when available; otherwise the demo states "demo fixture." Crop and scheme agents retrieve from a small official corpus. The final display contains:

- Evidence-backed field action, or a clear statement that evidence is insufficient.
- Mandi trend with observation dates and appropriately cautious timing guidance.
- Relevant scheme passage and eligibility status based on supplied facts.
- Safety warnings, stale/missing evidence, uncertainty, sources, and qualified-professional disclaimer.
- Append-only trace of agent execution and Python tool calls.

14. Risks and fallback strategies

Risk	Fallback
Framework API movement / compatibility	Early integration spike; pin working release; isolate domain logic. Seek approval before any framework change.
API access or incomplete coverage	Provider adapters and deterministic fixtures; label source status; do not invent data.
Ambiguous district or commodity	Controlled mappings and clarification/error state; never guess.
Incomplete or conflicting crop advice	Limit demos to curated combinations; show limitations or withhold the recommendation.
Scheme changes or unclear eligibility	Cite dated text; use unknown; refer to official administering authority.
LLM unsupported claims	Deterministic calculations and criteria; require source IDs; coordinator suppresses unsupported statements.
Ten-day schedule pressure	Complete one coherent all-agent demonstration before expanding geographic/crop coverage or UI polish.

15. Decisions awaiting approval

- Model/provider for interpretation and wording, or deterministic output until credentials are available.
- First curated crop and state/district, or permission to choose a practical documented example.
- Live IMD/OGD integrations when verified with fixtures as fallback (recommended), or fixture-only for the first submission.
- FastAPI web interface and TF-IDF RAG approach.

Planning boundary

This report proposes an implementation. It does not create or modify application files, install dependencies, initialize Git, add voice/multilingual generation, or change the selected architecture. Implementation should begin only after the unresolved decisions are approved.

References and verification sources

Microsoft Agent Framework: get started | Workflow concepts | Function tools | OGD / AGMARKNET catalog | IMD API reference | IMD API portal | Pydantic on PyPI | FastAPI on PyPI | HTTPX on PyPI | pytest on PyPI